ExploratoryGenius

```
# Load in jsonlite package
library(jsonlite)
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(igraph)
```

```
Attaching package: 'igraph'

The following objects are masked from 'package:dplyr':

    as_data_frame, groups, union

The following objects are masked from 'package:stats':

    decompose, spectrum
```

```
The following object is masked from 'package:base':

    union
```

```
#Assign Node Data and get a snapshot

#datausers_nominal <- stream_in(file("/Users/alexgould/Desktop/STA585GeniusProject/genius-exp
#head(datausers)
#data_annotate <- stream_in(file(("//Users/alexgould/Desktop/STA585GeniusProject/genius-exper
#head(data_annotate)
#songinfo <-  stream_in(file(("/Users/alexgould/Desktop/STA585GeniusProject/genius-expertise/
#head(songinfo)
# save(datausers_nominal, data_annotate, songinfo, file = "/Users/alexgould/Desktop/STA585Ge

library(igraph)

# Load in Node data
load("/Users/alexgould/Desktop/STA585GeniusProject/initialfiles.Rdata")

# Assign Edge Data
edges_nominal <- read.csv("/Users/alexgould/Desktop/STA585GeniusProject/genius-expertise/foll

valid_users <- intersect(
  unique(data_annotate$user[!is.na(data_annotate$user)]),
  intersect(
    intersect(unique(datausers_nominal$url_name),
              unique(edges_nominal$follower)),
    unique(edges_nominal$following)
  )
)

edges_reduced <- edges_nominal[edges_nominal$follower %in% valid_users &
                                 edges_nominal$following %in% valid_users,
                               c("follower", "following")]

valid_nodes    <- datausers_nominal[datausers_nominal$url_name %in% valid_users, ]
valid_annotate <- data_annotate[data_annotate$user %in% valid_users, ]

## ---- CHANGED: one genre per user from ALL of their annotations ----

# Song-level tag lookup (first listed tag for each song)
top_tag <- function(x) {
```

```r
  x <- unlist(x)
  if (length(x) == 0) NA_character_ else x[1]
}
song_tags <- data.frame(
  key = tolower(songinfo$url_name),
  tag = vapply(songinfo$tags, top_tag, character(1)),
  stringsAsFactors = FALSE
)
song_tags <- song_tags[!duplicated(song_tags$key), ]

# Tag every annotation of the valid users
valid_annotate$tag <- song_tags$tag[match(tolower(valid_annotate$song), song_tags$key)]
mean(is.na(valid_annotate$tag))        # share of annotations with no genre
```

```
[1] 0.001255913
```

```r
# Most frequent tag per user, ties broken at random (reproducible)
mode_tag <- function(x) {
  x <- x[!is.na(x)]
  if (length(x) == 0) return(NA_character_)
  tb  <- table(x)
  top <- names(tb)[tb == max(tb)]
  if (length(top) == 1) top else sample(top, 1)
}
set.seed(585)
user_tag <- tapply(valid_annotate$tag, valid_annotate$user, mode_tag)
user_tags <- data.frame(url_name = names(user_tag),
                        tag = as.character(user_tag),
                        stringsAsFactors = FALSE)

# One profile row per user, then attach the tag
nodes_one     <- valid_nodes[!duplicated(valid_nodes$url_name), ]
nodes_reduced <- merge(nodes_one, user_tags, by = "url_name", all.x = TRUE)

stopifnot(!anyDuplicated(nodes_reduced$url_name))
nrow(nodes_reduced)                    # should equal length(valid_users)
```

```
[1] 5559
```

```r
nrow(nodes_reduced) == length(valid_users) #TRUE
```

```
[1] TRUE
```

```r
sum(is.na(nodes_reduced$tag))          # users with no usable genre
```

```
[1] 5
```

```r
sort(table(nodes_reduced$tag), decreasing = TRUE)[1:10]
```

```
       Rap        R&B       Trap West Coast     France        Pop       Rock
      3112        284        212        146        138        118         81
French Rap    Atlanta East Coast
        80         76         75
```

```r
## ------------------------------------------------------------------------

# Drop self-loops and duplicate rows
edges_final <- edges_reduced[edges_reduced$follower != edges_reduced$following, ]
edges_final <- unique(edges_final)

# Keep only nodes with at least one remaining edge
keep_ids    <- unique(c(edges_final$follower, edges_final$following))
nodes_final <- nodes_reduced[nodes_reduced$url_name %in% keep_ids, ]

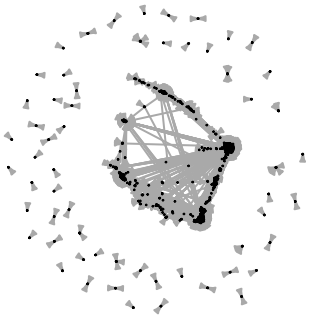
stopifnot(all(unlist(edges_final) %in% nodes_final$url_name))
stopifnot(all(nodes_final$url_name %in% keep_ids))
stopifnot(!anyDuplicated(nodes_final$url_name))

# Split up stats data frame
nodes_final$annotations    <- nodes_final$stats$Annotations
nodes_final$suggestions    <- nodes_final$stats$Suggestions
nodes_final$transcriptions <- nodes_final$stats$Transcriptions
nodes_final$questions      <- nodes_final$stats$Questions
nodes_final$answers        <- nodes_final$stats$Answers
nodes_final$pyongs         <- nodes_final$stats$Pyongs
nodes_final <- subset(nodes_final, select = -stats)
```

```r
# Roles to boolean variables
all_roles <- sort(unique(unlist(nodes_final$roles)))
for (r in all_roles) {
  col <- paste0("role_", make.names(r))
  nodes_final[[col]] <- vapply(nodes_final$roles, function(x) r %in% x, logical(1))
}
nodes_final <- subset(nodes_final, select = -roles)

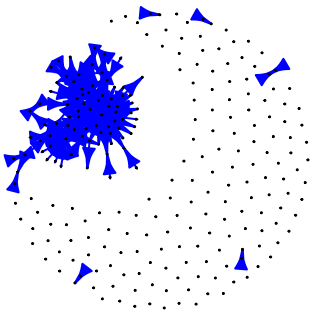
# Build graph
g <- graph_from_data_frame(d = edges_final, vertices = nodes_final, directed = TRUE)

plot(g, vertex.label = NA, vertex.size = 0.5, edge.arrow.size = 0.2)
```



G(n,p) graph visualization

```r
# Small standalone graph (no copy of the big one)
sampled_nodes <- sample(V(g),250 )
induced <- induced_subgraph(g,sampled_nodes)
E(induced)$color <- "blue"
plot(induced, vertex.label = NA, layout = layout_with_fr(induced),
     vertex.size = 0.7, edge.arrow.size = 0.5) #Not pretty
```



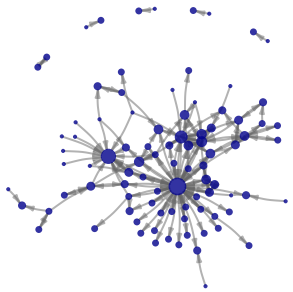
```r
#Number of Isolates
n_iso <- sum(degree(induced) == 0)

# Suppress Isolates from this visualization
g_plot <- delete_vertices(induced, which(degree(induced) == 0))

set.seed(00)
lay <- layout_with_fr(g_plot)

viz <- plot(g_plot,
     layout = lay,
     vertex.label = NA,
     vertex.size = 3 + 2 * sqrt(degree(g_plot, mode = "in")),   # size by in-degree (popular
     vertex.color = adjustcolor("darkblue", 0.8),
     vertex.frame.color = NA,
     edge.arrow.size = 0.2,
     edge.arrow.width = 0.8,
     edge.color = adjustcolor("grey40", 0.5),
     edge.curved = 0.1,
     main = "Induced Subgraph of 250 Randomly Sampled Nodes")
title(sub = paste(n_iso, "isolated nodes omitted"))
```

Induced Subgraph of 250 Randomly Sampled Nodes

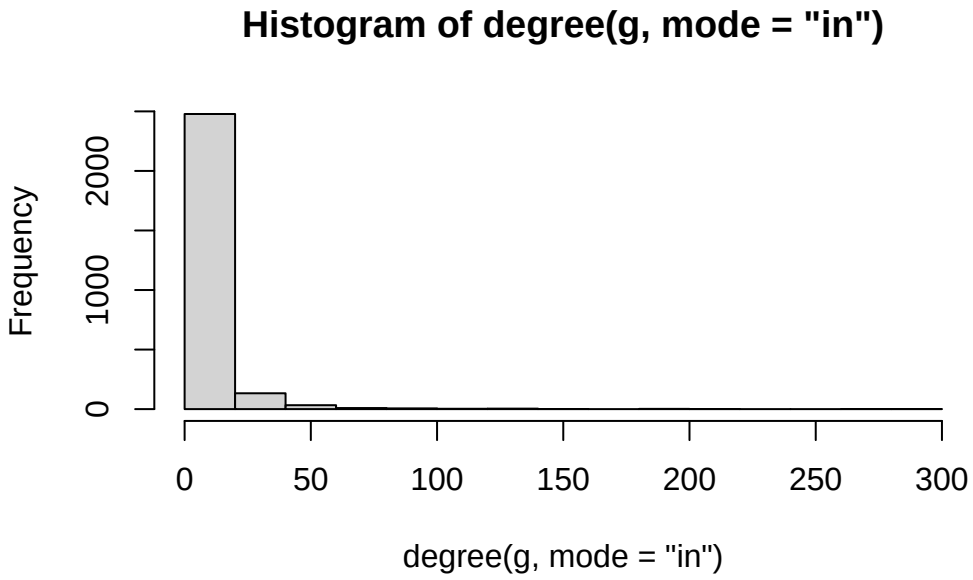


159 isolated nodes omitted

```r
save(viz, file = "/Users/alexgould/Desktop/STA585GeniusProject/inducedviz.png")
```

Descriptive Analysis

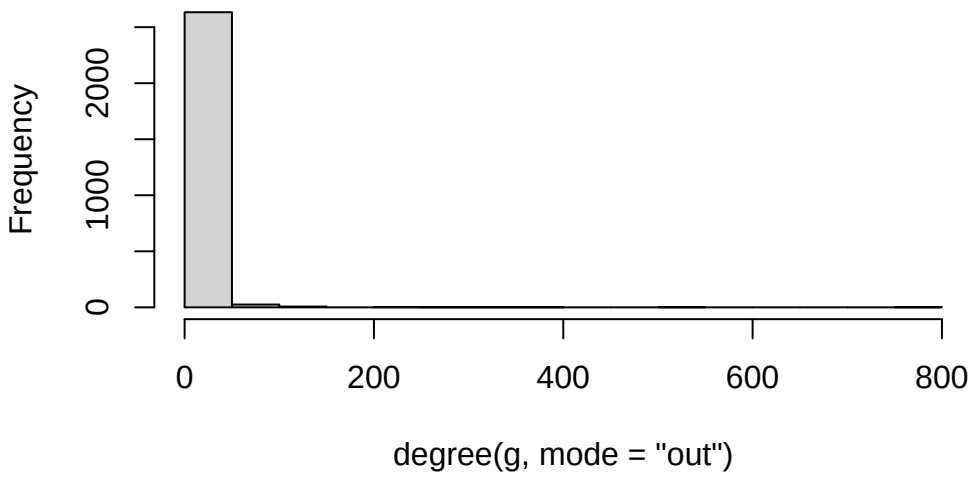
```r
# A. Node Degree Distribution
hist(degree(g, mode = "in"))
```




```r
table(degree(g, mode = "in"))
```

```
  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19
481 882 359 178 121  69  56  49  44  30  39  23  22  22  20  19  11  14  15  15
 20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39
  9  14  14  12   9   4  15   5  12   7   6   4   3   3   2   2   4   4   3   6
 40  41  42  43  45  46  47  48  49  50  51  52  53  55  56  57  59  60  62  64
  4   5   1   1   1   5   1   3   1   1   2   1   2   2   3   1   1   2   3   1
 68  70  71  74  83  84  89  92  95  97 103 108 116 125 126 135 140 158 194 199
  2   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
208 266 285
  1   1   1
```

```r
hist(degree(g, mode = "out"))
```




```
table(degree(g, mode = "out"))
```

```
  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19
720 807 280 162  98  73  53  48  40  31  23  24  19  19  16  17  14  17  14  10
 20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39
 13  14   8   7   8   9   6   5  16   7   5   6   7   2   3   2   2   1   2   2
 40  41  42  43  44  47  48  49  50  51  54  55  56  58  59  61  64  66  67  71
  2   5   3   3   4   1   1   3   1   1   2   1   2   1   2   1   1   1   4   2
 76  80  84  89  94  96 101 105 119 120 133 138 150 217 222 260 317 354 511 769
  2   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
```

```
mean(degree(g, mode = "in"))
```

```
[1] 5.912799
```

```
sd(degree(g, mode = "in"))
```

```
[1] 15.55619
```

```
mean(degree(g, mode = "out"))
```

```
[1] 5.912799
```

```r
sd(degree(g, mode = "out"))
```

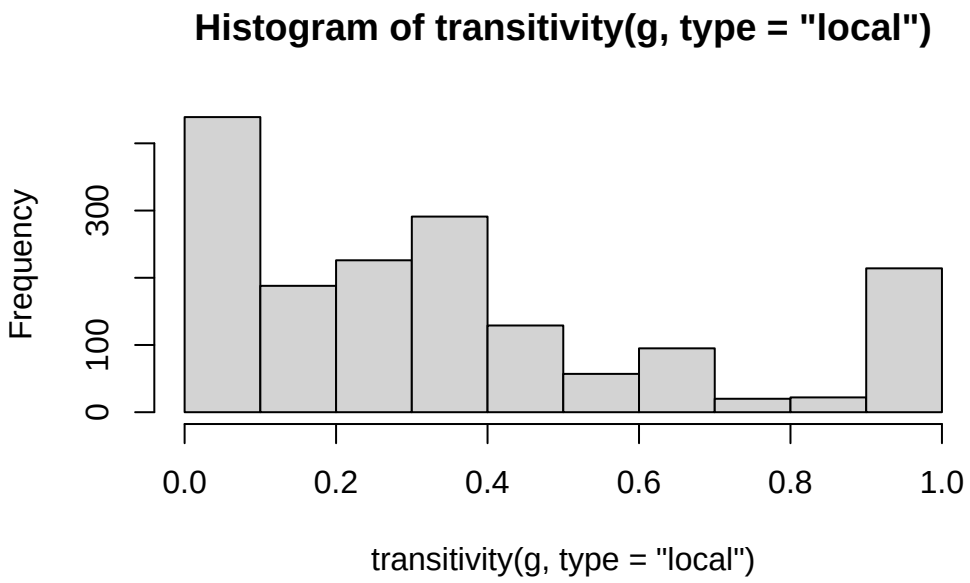
```
[1] 24.19126
```

```r
# B. Global and Local Density
#Global
edge_density(g)
```

```
[1] 0.002213703
```

```r
#Local


#C. Clustering Coefficient
hist(transitivity(g, type = "local"))
```




```r
transitivity(g, type = "global")
```

```
[1] 0.09051928
```

```r
mean(na.omit(transitivity(g, type = "local")))
```

```
[1] 0.3544141
```

```
sd(na.omit(transitivity(g, type = "local")))
```

```
[1] 0.3199898
```

```
# D: Average Betweenness
mean(betweenness(g))
```

```
[1] 3231.159
```

```
#E : Degree/Nominal Assortativity
#Degree Assortativity
assortativity_degree(g)
```

```
[1] -0.1929364
```

```
#Nominal Assortativity


#F: Graph Partinitioning
#cluster_fast_greedy(g)
#Only works on undirected graphs
```